

ARROWSTACK CYBER SECURITY INTERNSHIP

TASK 1 — CRYPTOGRAPHIC ENCRYPTION & HASHING SUITE

AES-256-GCM • RSA-2048-OAEP • SHA-256

Student	Abdul Rehman
Internship	Cyber Security — Intermediate / Technical Track
Task	Cryptographic Encryption & Hashing Suite (AES, RSA, SHA-256)
Date	1 October 2026
Status	Completed and locally validated

1. Executive Summary

This project implements a Python-based educational cryptographic suite demonstrating AES-256-GCM symmetric authenticated encryption, RSA-2048-OAEP asymmetric encryption, and SHA-256 one-way hashing. The implementation was executed in a local Windows laboratory environment using test messages only. Automated self-testing completed successfully for all three components.

2. Objective

The objective was to implement and validate a Python-based cryptographic suite demonstrating AES symmetric encryption, RSA asymmetric encryption, and SHA-256 hashing.

3. Scope

The work was performed in an educational/laboratory environment. The demonstrations used test messages and did not use production credentials, organizational secrets, or real confidential data.

4. Environment

Item	Observed environment
Operating system	Microsoft Windows
Development environment	Visual Studio Code
Python	Python 3.13.x environment (cp313 package observed)
Cryptography library	cryptography 46.0.7
Dependency specification	cryptography>=42, <47
Execution environment	Python virtual environment (.venv)

5. Implementation

5.1 AES-256-GCM

AES was implemented as symmetric authenticated encryption using a 256-bit key. The program generates a random key and a random 12-byte nonce for each encryption operation, encrypts the plaintext with AES-GCM, and then decrypts the result to verify that the original plaintext is recovered.

5.2 RSA-2048-OAEP

RSA was implemented using a 2048-bit key pair. The public key is used for encryption and the private key for decryption. OAEP padding is configured with SHA-256.

5.3 SHA-256

SHA-256 was used to generate a fixed-length 256-bit cryptographic digest. SHA-256 is a one-way hash and is not reversible encryption.

Test input: Arrowstack cybersecurity lab

Observed digest: b7027c088b31e96ecad073603456f589bf30f7d0062de5b5da3ecc8be28fb983

6. Automated Validation

The following command was executed: `python crypto_suite.py --self-test`

```
=== Cryptographic Suite Self-Test ===  
[PASS] AES-GCM encryption/decryption  
[PASS] RSA-OAEP encryption/decryption  
[PASS] SHA-256 hashing
```

All self-tests passed.

Component	Test	Result
AES-256-GCM	Encrypt → decrypt → compare original	PASS
RSA-2048-OAEP	Public-key encrypt → private-	PASS

	key decrypt	
SHA-256	Generate 64-character hexadecimal digest	PASS

7. Evidence

The evidence below records the actual local demonstrations. Public copies have been sanitized to avoid exposing local filesystem paths and the ephemeral AES demonstration key.

Evidence 1 — Automated self-test

```
Local path redacted for public evidence

suite.py --self-test
=== Cryptographic Suite Self-Test ===
[PASS] AES-GCM encryption/decryption
[PASS] RSA-OAEP encryption/decryption
[PASS] SHA-256 hashing

All self-tests passed.
(.venv) PS C:\Users\alman\Downloads\Arrowstack_Tasks_1_2_Complete_Solution\Arrowstack_Complete_Solution\Arrowstack_Tasks_1_2\Task-1-Cryptographic-Encryption-Hashing-Suite>
```

Evidence 2 — SHA-256 demonstration

```
Local path redacted for public evidence

(.venv) PS C:\Users\alman\Downloads\Arrowstack_Tasks_1_2_Complete_Solution\Arrowstack_Complete_Solution\Arrowstack_Tasks_1_2\Task-1-Cryptographic-Encryption-Hashing-Suite> suite.py --hash "Arrowstack cybersecurity lab"
SHA-256: b7027c088b31e96ecad073603456f589bf30f7d0062de5b5da3ecc8be28fb983
(.venv) PS C:\Users\alman\Downloads\Arrowstack_Tasks_1_2_Complete_Solution\Arrowstack_Complete_Solution\Arrowstack_Tasks_1_2\Task-1-Cryptographic-Encryption-Hashing-Suite>
```

Evidence 3 — AES-256-GCM demonstration

```
(.venv) PS C:\Users\alman\Downloads\Arrowstack_Tasks_1_2_Complete_Solution\Arrowstack_Complete_Solution\Arrowstack_Tasks_1_2\Task-1-Cryptographic-Encryption-Hashing-Suite> suite.py --hash "Arrowstack cybersecurity lab"
SHA-256: b7027c088b31e96ecad073603456f589bf30f7d0062de5b5da3ecc8be28fb983
(.venv) PS C:\Users\alman\Downloads\Arrowstack_Tasks_1_2_Complete_Solution\Arrowstack_Complete_Solution\Arrowstack_Tasks_1_2\Task-1-Cryptographic-Encryption-Hashing-Suite> python crypt.py
AES-GCM key (redacted for report)

AES-GCM key (Base64): Wwsla25SAH0NJZBFf8d/O3WxR6SLxiwZf6xV3twXr4E=
Ciphertext (Base64): a3GmtI1KF8kfnzvblfvipIsnkUz7u/YWtnf9M5mYZEGFkYc4Ki3J2VP1Js1/YPQ/TCGcNg
Decrypted: Confidential lab message
(.venv) PS C:\Users\alman\Downloads\Arrowstack_Tasks_1_2_Complete_Solution\Arrowstack_Complete_Solution\Arrowstack_Tasks_1_2\Task-1-Cryptographic-Encryption-Hashing-Suite>
```

Evidence 4 — RSA-2048-OAEP demonstration

```
Local path redacted for public evidence

(.venv) PS C:\Users\alman\Downloads\Arrowstack_Tasks_1_2_Complete_Solution\Arrowstack_Comple
e_Solution\Arrowstack_Tasks_1_2\Task-1-Cryptographic-Encryption-Hashing-Suite> python crypto
● suite.py --aes-demo "Confidential lab message"
AES-GCM key (Base64): WwS1a25SAH0NJZBFf8d/O3WxR6SLxiwZf6xV3twXr4E=
Ciphertext (Base64): a3GmtI1KF8kfzvbLfvipIsnkuZ7u/YWtnf9M5mYZEGfkYc4Ki3J2VP1Js1/YPQ/TCGcNg=
Decrypted: Confidential lab message
(.venv) PS C:\Users\alman\Downloads\Arrowstack_Tasks_1_2_Complete_Solution\Arrowstack_Comple
e_Solution\Arrowstack_Tasks_1_2\Task-1-Cryptographic-Encryption-Hashing-Suite> python crypto
● suite.py --rsa-demo "RSA lab message"
RSA-OAEP ciphertext (Base64): h44lqT8F/ySgYTvra/dhz+/RJDZU0/cu5KY9y2YHnk1UQkKsyUNwJFT126G2Sz
1iwyuAH/W3vRJ7tr6tYkEsLlvG5o9RJC5SKKY9Z4jppC7k1lXzz8iP1ZGLEddb99e0dbNtMo+r0JYI1IrsSCUjvMND1p
lv8GdBXqDJHQw79bt+bV/8c2FFASFYGG13gPy/jGhC1ks4SHT6dBVSwpKj6jGLbxnBDDz91pILAuIDqvuwPHGZPBZBV+
u9oYrytMsKvm084YXgzoM0NwxEfILsJOEHAwhVWg9hErPKrycu2me5T3X1/lvFI9M/IcrflouA1XAebfZMwFx2L+ngSF
==
Decrypted: RSA lab message
○ (.venv) PS C:\Users\alman\Downloads\Arrowstack_Tasks_1_2_Complete_Solution\Arrowstack_Comple
```

8. Security Considerations

- Private RSA keys must never be committed to public source control.
- The project .gitignore excludes keys/private_key.pem.
- AES-GCM provides authenticated encryption and is preferred over unauthenticated AES modes for application data.
- RSA-OAEP is used instead of textbook RSA encryption.
- SHA-256 alone is not suitable for password storage; password storage should use a dedicated password hashing/KDF.
- Real credentials, tokens, passwords, private keys, and confidential data should not be included in screenshots or public repositories.
- RSA is demonstrated with short text; real applications normally use hybrid encryption for larger data.

9. Repository Security

The project .gitignore excludes keys/private_key.pem, __pycache__/, *.pyc, and .venv/. The private RSA key generated during local testing must remain local and must not be uploaded to GitHub.

10. Limitations

This is an educational demonstration rather than a production cryptographic platform. It does not implement enterprise key rotation, hardware-backed key storage, certificate management, centralized key management, production secret-management infrastructure, or a complete access-control policy.

11. Reproducibility

```
python -m venv. venv
```

```
. venv\Scripts\activate
```

```
pip install -r requirements.txt
```

```
python crypto_suite.py --self-test
```

```
python crypto_suite.py --hash "Arrowstack cybersecurity lab"
```

```
python crypto_suite.py --aes-demo "Confidential lab message"
```

```
python crypto_suite.py --rsa-demo "RSA lab message"
```

12. Conclusion

The Arrowstack Task 1 cryptographic suite was implemented and tested successfully. AES-256-GCM demonstrated symmetric authenticated encryption, RSA-2048-OAEP demonstrated asymmetric encryption and decryption, and SHA-256 demonstrated one-way hashing. The automated self-test produced PASS results for all three required components. The project is ready for GitHub publication after confirming that no private key, secret, credential, or other sensitive information is included in the repository.

Appendix A — Submission Checklist

- ☒ AES-256-GCM implemented
- ☒ RSA-2048-OAEP implemented
- ☒ SHA-256 implemented
- ☒ Automated self-test passed
- ☒ SHA-256 demonstration completed
- ☒ AES demonstration completed
- ☒ RSA demonstration completed
- ☒ README prepared
- ☒ Requirements file prepared
- ☒ .gitignore protects the RSA private key
- ☒ Evidence screenshots prepared